

13. ONE-PAGE WRITE-UP

NV-CES: Transaction and Concurrency Management

Introduction

NV-CES is a database-based examination management system designed to handle examination slots, candidate bookings and examination results. A major challenge in such a system is ensuring data consistency when multiple users access the same database records simultaneously. Module 2 addresses this problem through transaction management and concurrency control.

Problem

Without transaction control, two candidates may attempt to book the last available examination seat at the same time. Both transactions could read the same available capacity and create conflicting booking records. Similarly, two examiners updating the same candidate's result could overwrite one another's changes. These situations can lead to overbooking, incorrect result data and loss of important updates.

Proposed Solution

The system uses **pessimistic concurrency control** with MySQL transactions and row-level locking. The SELECT ... FOR UPDATE statement locks the required row before modification. The transaction then performs the required operations and completes using COMMIT. If an error occurs, ROLLBACK is used to remove the uncommitted changes.

For slot booking, the system first locks the selected row in exam_slots and checks whether seats are available. If capacity is available, the booking is inserted and booked_count is incremented before committing the transaction.

For result processing, the system locks the result record before updating it. The score and version number are read, the new score is stored, the version is incremented and the change is inserted into result_update_history.

Concurrency Demonstration

The implementation was tested using two independent MySQL sessions. During the booking experiment, one session locked an examination slot while the second session attempted to access the same row. The second session waited until the first transaction committed. It then observed the updated booking count.

The result concurrency experiment started with:

Score = 55.00

Version = 0

The first examiner updated the score to:

58.00

Version = 1

After the first transaction committed, the second examiner obtained the lock and observed the updated score. The second transaction then changed the score to:

60.00

Version = 2

The final live result was therefore **60.00**, while the history table preserved both changes: **55.00** → **58.00** and **58.00** → **60.00**.

Results

The implementation successfully demonstrated transaction atomicity, row-level locking, controlled concurrent access, rollback and version tracking. The booking test ensured that the slot count remained within its capacity. The result test ensured that concurrent examiner updates were serialized instead of operating on stale data.

Conclusion

Module 2 demonstrates how DBMS transaction and concurrency-control concepts can be applied to a practical examination system. The use of transactions, SELECT ... FOR UPDATE, COMMIT, ROLLBACK, SAVEPOINT, version tracking and audit history provides a reliable mechanism for maintaining consistency and accountability. The implementation successfully addresses the major concurrency problems associated with examination-slot booking and result processing.

